Working with Conda

Conda, being a powerful tool, can create environments and can deal with packages. Hence, it is called both an environment manager and a package manager. The general syntax of a Conda command is as follows:

```
conda [-h] [-V] command.....
```

… where *-h* is help and *–V* gives the Conda version installed in our system.

The following is a list of Conda commands.

- *info:* Displays current Conda install details including platform, Conda version, Python version, root environment, environment directories, channel URLs and configuration file.
- *list:* Displays the list of installed packages in a Conda environment.
- *help:* Shows the list of Conda commands and their options. For example:

```
conda list -h
```

…displays the options available for the *list* command.

- *search:* Displays a list of packages matching the search string.
- *create:* Creates a virtual environment for the user to work with.
- *install:* Installs the specified packages to the Conda environment.
- *upgrade:* Updates the installed packages to the latest compatible versions.
- *remove:* Removes the specified packages from the Conda environment.
- *config: .condarc* can be modified using this command.
- *clean:* Removes unused packages and caches.

Creating an environment

When you want to experiment with packages, but don't know their side effects on the system configuration, or you have an application that needs a package version different from the version you have already installed in your system (and the already installed version is needed for working of some other applications), you can create an environment other than the root environment. Such an environment will be virtual, and will work using the existing resources of the platform on which it is created.

The following command will create an environment *env_name* with no specific packages installed in that environment:

```
conda create --name env_name
```

You can alternatively use *-n* for *--name*. You can install packages in the environment at the time of creation by modifying the above command as follows:

```
conda create -n env_name list_of_packages
```

If you want to install a specific version of a package, you can specify a version number along with the package name. For example, *Python 3.4.6* and *numpy 1.2* can be installed in an environment named 'py' at the time of its creation using the following command:

```
conda create -n py python=3.4.6 numpy=1.2
```

It is worth noting that the environment created using Conda is isolated, but not in every sense. Consider the following scenario. You have installed Python 2.7.12 based Miniconda in your system. You are creating a virtual environment without specifying any packages. You may think that, in this condition, a newly created environment cannot serve any purpose because it does not contain any packages. But while installing Miniconda, packages like Python and its dependencies are automatically installed, and the Conda environment created has access to these packages. It has access to the root directory also. In this scenario, if you want to install any other version of Python in the virtual environment, it is advisable to install the package at the time of the creation of the environment itself. The same is true about any package being installed with Miniconda.

Once a virtual environment is created, the command to activate that environment will be automatically displayed in the terminal. In Linux based systems, you can activate it using the following command:

```
source activate env_name
```

Installation of packages

Being a package manager, Conda can be used to install or uninstall packages. Packages can be installed in the current environment using the following command:

```
conda install package_list
```

If you want to install packages in an environment other than the current environment, use the following command:

```
conda install -n env_name package_list
```

As stated earlier, versions can be specified along with the package name.

Other package managers like *apt-get* and *pip* can also be used to install packages in a Conda environment. At times, when Conda fails, *pip* may succeed. This is because some Python packages not available in the Conda repository are